

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 1

Chapter 2: Application Layer

Our goals:

- ❑ conceptual, implementation aspects of network application protocols
 - ❖ transport-layer service models
 - ❖ client-server paradigm
 - ❖ peer-to-peer paradigm
- ❑ programming network applications
 - ❖ sockets API
- ❑ learn about protocols by examining popular application-level protocols
 - ❖ DNS
 - ❖ FTP
 - ❖ HTTP
 - ❖ SMTP
 - ❖ P2P

9/30/2013

Application Layer (SSL) 2

Some network apps

- ❑ remote login
- ❑ e-mail, ftp
- ❑ web
- ❑ instant messaging
- ❑ P2P file sharing
- ❑ multi-user network games
- ❑ streaming stored video
- ❑ voice over IP
- ❑ real-time video conferencing
- ❑ social networking
- ❑ cloud computing
 - ❖ data center networks
- ❑
- ❑

9/30/2013

Application Layer (SSL) 3

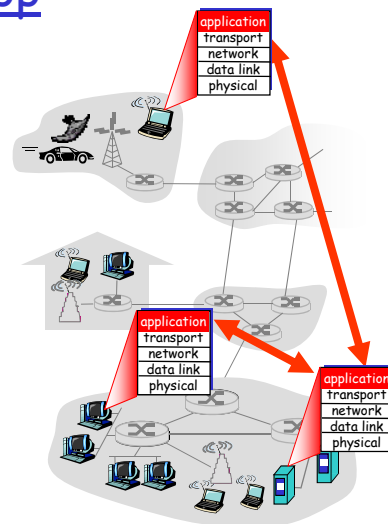
Creating a network app

Write programs that

- ❖ run on (different) *end systems*
- ❖ communicate over network
- ❖ e.g., web server software communicates with browser software

No need to write software for network-core devices

- ❖ Network-core devices do not run user applications
- ❖ This approach allows for **rapid app development**, propagation



9/30/2013

Application Layer (SSL) 4

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 5

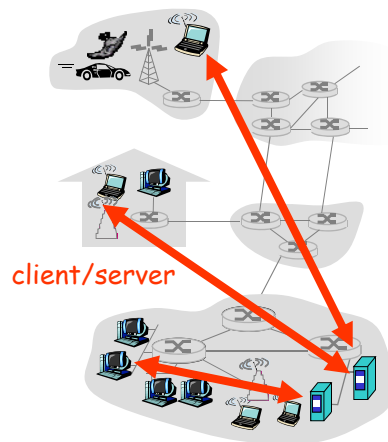
Application architectures

- ❑ Client-server
- ❑ Peer-to-peer (P2P)
- ❑ Hybrid of client-server and P2P

9/30/2013

Application Layer (SSL) 6

Client-server architecture



9/30/2013

server:

- ❖ always-on host
- ❖ permanent IP address
- ❖ data centers for scaling

clients:

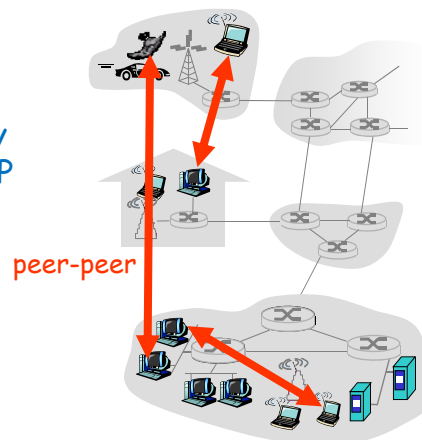
- ❖ communicate with server (*speak first*)
- ❖ may be intermittently connected
- ❖ may have dynamic IP addresses
- ❖ do not communicate directly with each other

Application Layer (SSL) 7

Pure P2P architecture

- ❑ no always-on server
- ❑ arbitrary end systems directly communicate
- ❑ peers are intermittently connected and change IP addresses

Difficult to manage (e.g., find by flooding)



9/30/2013

Application Layer (SSL) 8

Hybrid of client-server and P2P

Example: Skype

- ❖ voice-over-IP P2P application
- ❖ centralized server: finding address of remote party
- ❖ client-client connection: direct (not through server)
 - NAT traversal problem

9/30/2013

Application Layer (SSL) 9

Processes communicating

Process: program running within a host.

- within same host, two processes communicate using **inter-process communication** (defined by OS).
- processes in different hosts communicate by exchanging **messages**

Client process: process that initiates communication

Server process: process that waits to be contacted

- Note: client & server processes run in P2P as well as client-server apps

Note: A process in these slides may be a thread instead

9/30/2013

Application Layer (SSL) 10

Addressing processes

- ❑ to receive messages, process must have an *identifier*
- ❑ host device has unique 32-bit IP address
- ❑ *Q:* is IP address of host on which process runs enough for identifying the process?
 - ❖ *A:* No, many processes can be running on same host
- ❑ *identifier* includes both IP address and port numbers associated with process on host.
- ❑ Example port numbers:
 - ❖ HTTP server: 80
 - ❖ Mail server: 25
- ❑ to send HTTP message to gaia.cs.umass.edu web server:
 - ❖ IP address: 128.119.245.12
 - ❖ Port number: 80
- ❑ more shortly...

9/30/2013

Application Layer (SSL) 11

App-layer protocol defines

- ❑ Types of messages exchanged,
 - ❖ e.g., request, response
 - ❑ Message syntax:
 - ❖ what fields in messages & how fields are delineated
 - ❑ Message semantics
 - ❖ meaning of information in fields
 - ❑ Rules for when and how processes send & respond to messages
- Public-domain protocols:**
- ❑ defined in RFCs which enable *interoperability* e.g., HTTP, SMTP
- Proprietary protocols:**
- ❑ e.g., Skype

9/30/2013

Application Layer (SSL) 12

What transport service does an app need?

Data loss

- some apps (e.g., audio) can tolerate some loss
- other apps (e.g., file transfer, telnet) require 100% reliable data transfer

Timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

Throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

Security

- Encryption, data integrity, ... (app can do this)

9/30/2013

Application Layer (SSL) 13

Transport service requirements of common apps

Application	Data loss	Throughput	Time Sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic (few kbps)	no
interactive audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	yes, 100s msec
streaming stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100s msec
instant messaging	no loss	elastic	yes and no

9/30/2013

Application Layer (SSL) 14

Internet transport protocols services

TCP service:

- ❑ *connection-oriented*: setup required between client and server processes
- ❑ *reliable transport*: between sending and receiving process
- ❑ *flow control*: sender won't overwhelm receiver
- ❑ *congestion control*: throttle sender when network is overloaded
- ❑ *does not provide*: timing, minimum throughput guarantees, security

UDP service:

- ❑ *unreliable data transfer* between sending and receiving processes
- ❑ *does not provide*: connection setup, reliability, flow control, congestion control, timing, throughput guarantee, or security

Q: why bother? Why is there a UDP?

9/30/2013

Application Layer (SSL) 15

Internet apps: application & transport protocols

Application	Application layer protocol	Underlying transport protocol
e-mail	SMTP [RFC 5321]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e. g., Youtube)	TCP
Internet telephony	SIP[RFC 3261], RTP[RFC 3550], proprietary(e.g., Skype)	UDP or TCP

9/30/2013

Application Layer (SSL) 16

Chapter 2: Application layer

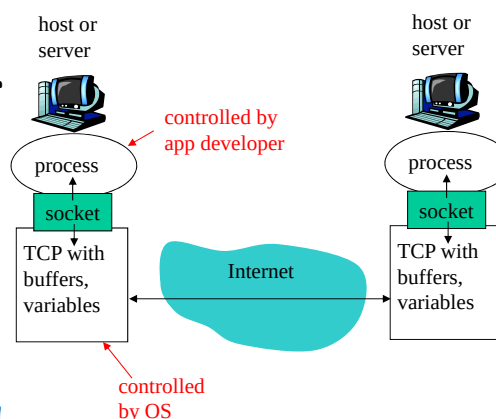
- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 **Socket programming with TCP**
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 17

Sockets

- ❑ process sends/receives messages to/from its **socket**
- ❑ socket analogous to door
 - ❖ sending process shoves message out door
 - ❖ it relies on transport infrastructure on other side of door which brings message to socket at receiving process



9/30/2013

Application Layer (SSL) 18

Socket programming

Goal: learn how to write client and server programs which communicate by sending data into sockets, reading data out of sockets

Socket API

- ❑ introduced in BSD4.1 UNIX, 1981
- ❑ sockets are explicitly created, used, then released by applications
- ❑ client/server paradigm
- ❑ API: choice of a transport protocol and ability to specify a few parameters
 - ❖ reliable byte stream
 - ❖ unreliable datagram

9/30/2013

Application Layer (SSL) 19

Socket programming *with TCP*

Before client contact:

- ❑ server process must first be running
- ❑ server must have created socket (door) that welcomes client's contact

Client contacts server by:

- ❑ creating client-local TCP socket
- ❑ specifying IP address, port number of server process
- ❑ When **client makes connect call**: client TCP establishes connection to server TCP

- ❑ When contacted by client, **server TCP creates a new connection socket** for server process to communicate with client

- ❖ allows server to talk with multiple clients
- ❖ source port numbers used to distinguish clients (*more in Chap 3*)

application viewpoint

TCP provides reliable, in-order transfer of byte stream between client and server

9/30/2013

Application Layer (SSL) 20

Socket API for TCP (BSD Unix)

❑ Client side

- ❖ `socket()`, returns client socket id
- ❖ `connect()`, need to specify server IP address and port, sends conn req
- ❖ `send()`, sends to client socket
- ❖ `recv()`, receives from client socket
- ❖ `close()`, closes connection

- ❑ note: OS supplies local IP address and port for client

9/30/2013

❑ Server side

- ❖ `socket()`, returns server socket id
- ❖ `bind()`, binds server socket to server IP address and port
- ❖ `listen()`, willing to accept conn req on server socket
- ❖ `accept()`, accepts new conn req and returns its connection socket id
- ❖ `recv()`, receives from connection socket
- ❖ `send()`, sends to connection socket
- ❖ `close()`, closes connection

Application Layer (SSL) 21

Client/server socket interaction: TCP

server (running on `hostid`)

client

create socket,
port=`x`, for incoming
request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket =
serverSocket.accept()`

read request from
`connectionSocket`

write reply to
`connectionSocket`

close
`connectionSocket`

create socket
`clientSocket = socket()`
connect to `hostid`, port=`x`

send request using
`clientSocket`

read reply from
`clientSocket`

close
`clientSocket`

--- TCP ---
connection setup

9/30/2013

Application Layer (SSL) 22

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 23

Socket programming *with UDP*

UDP: no "connection" between client and server

- ❑ no handshaking
- ❑ sender explicitly attaches IP address and port of destination to each packet
- ❑ server must extract IP address, port of sender from received packet

UDP: transmitted data may be received out of order, or lost

application viewpoint

UDP provides unreliable transfer of groups of bytes ("datagrams") between client and server

9/30/2013

Application Layer (SSL) 24

Socket API for UDP (BSD Unix)

❑ Client side

- ❖ `socket()`, returns client socket id
- ❖ `sendto()`, sends to client socket, need to specify destination's IP address and port
- ❖ `recvfrom()`, receives from client socket: data and sender's IP address and port
- ❖ `bind()`, optional

- ❑ note: **needs timeout management**;
OS supplies local IP address and port for client if `bind()` not used

9/30/2013

❑ Server side

- ❖ `socket()`, returns server socket id
- ❖ `bind()`, binds server socket to server IP address and port
- ❖ `recvfrom()`, receives from server socket: data and sender's IP address and port
- ❖ `sendto()`, sends to server socket, need to specify destination's IP address and port

Application Layer (SSL) 25

Client/server socket interaction: UDP

server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET, SOCK_DGRAM)`

↓
read datagram from
`serverSocket`

↓
write reply to
`serverSocket`
specifying
client address,
port number

client

create socket:
`clientSocket =
socket(AF_INET, SOCK_DGRAM)`

↓
create datagram with server IP
address and port=x; send datagram
via `clientSocket`

↓
read datagram from
`clientSocket`
↓
close
`clientSocket`

9/30/2013

Application Layer (SSL) 26

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 27

DNS: Domain Name System

People: many identifiers:

- ❖ SSN, name, passport #

Internet hosts, routers:

- ❖ IP address (32 bit)
 - used for addressing datagrams
- ❖ "name", e.g.,
www.yahoo.com - used by humans

Required: map between name and IP addresses

Domain Name System:

- ❑ *distributed database*
implemented as a hierarchy of many *name servers*
- ❑ *application-layer protocol*
used to *resolve* names (address/name translation)
 - ❖ note: core Internet function, implemented as application-layer protocol
 - ❖ complexity at network's "edge"

9/30/2013

Application Layer (SSL) 28

DNS

DNS services

- ❑ Hostname to IP address translation
- ❑ Host alias
 - ❖ Canonical and alias names
- ❑ Mail server alias
- ❑ Load distribution
 - ❖ Replicated Web servers: set of IP addresses for one canonical name

Why not centralized DNS?

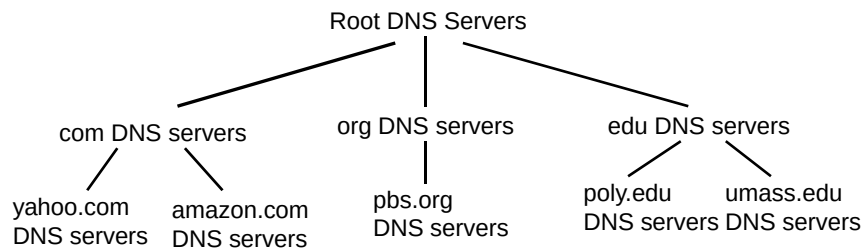
- ❑ single point of failure
- ❑ traffic volume
- ❑ distant centralized database
 - ❖ maintenance/update

doesn't *scale*!

9/30/2013

Application Layer (SSL) 29

Distributed, Hierarchical Database



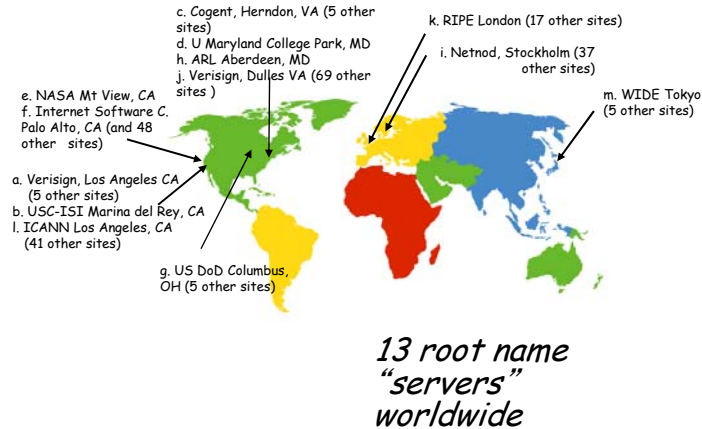
Client wants IP address for www.amazon.com; 1st approx:

- ❑ Client queries a root server to find .com DNS server
- ❑ Client queries .com DNS server to get amazon.com DNS server
- ❑ Client queries amazon.com DNS server to get IP address for www.amazon.com

9/30/2013

Application Layer (SSL) 30

DNS: root name servers



9/30/2013

Application Layer (SSL) 31

TLD and Authoritative Servers

- ❑ **Top-level domain (TLD) servers:** responsible for .com, .org, .net, .edu, ..., and all top-level country domains .uk, .fr, .ca, .jp
 - ❖ Verisign maintains servers for .com TLD
 - ❖ Educause for .edu TLD
- ❑ **Authoritative DNS servers:** organization's DNS servers, providing authoritative hostname-to-IP mappings for organization's named hosts (e.g., Web and mail).
 - ❖ Can be locally maintained by organization or a service provider

9/30/2013

Application Layer (SSL) 32

Local Name Server

- ❑ does not strictly belong to hierarchy
- ❑ each ISP (residential ISP, company, university) has one.
 - ❖ also called "default name server"
- ❑ when host makes DNS query, query is sent to its local DNS server
 - ❖ acts as proxy, forwards query into hierarchy

9/30/2013

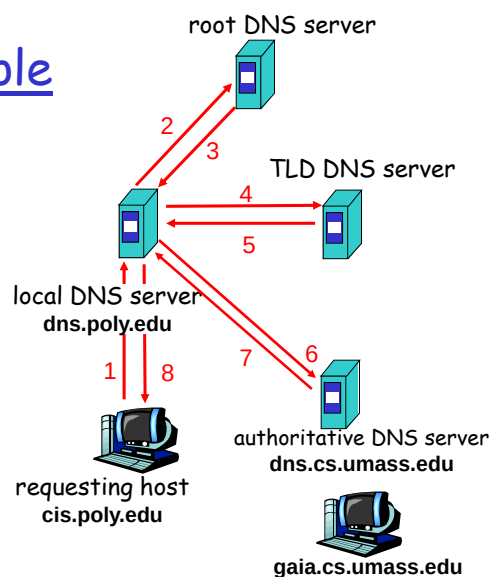
Application Layer (SSL) 33

DNS name resolution example

- ❑ Host at cis.poly.edu wants IP address for gaia.cs.umass.edu
- ❑ It sends a **recursive query** to local DNS server

iterated query:

- ❑ contacted server replies with name of server to contact
- ❑ "I don't know this name, but ask this server"



9/30/2013

Application Layer (SSL) 34

DNS: caching and updating records

- when (any) name server learns mapping, it *caches* mapping
 - ❖ cached entries timeout (disappear) after some time
 - ❖ TLD servers typically cached in local name servers -Thus root name servers not often visited
- cached entries may be *out-of-date* (best effort service)
 - ❖ if host changed IP address, this may not be known Internet-wide until all TTLs expire

9/30/2013

Application Layer (SSL) 35

DNS records

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

- | | |
|---|---|
| <ul style="list-style-type: none">□ Type=A<ul style="list-style-type: none">❖ name is hostname❖ value is IP address□ Type=NS<ul style="list-style-type: none">❖ name is domain (e.g. foo.com)❖ value is hostname of authoritative name server for this domain | <ul style="list-style-type: none">□ Type=CNAME<ul style="list-style-type: none">❖ name is alias name for some "canonical" (the real) name
e.g., www.ibm.com is really serveeast.backup2.ibm.com❖ value is canonical name□ Type=MX<ul style="list-style-type: none">❖ value is host name of mail server associated with name |
|---|---|

9/30/2013

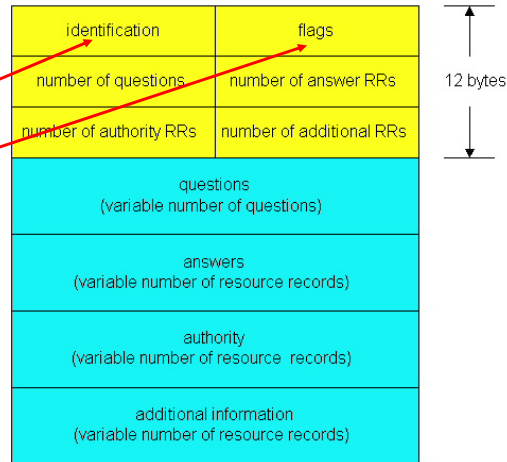
Application Layer (SSL) 36

DNS protocol, messages

DNS protocol : *query* and *reply* messages, both with same *message format*

msg header

- **identification**: 16 bit #
for query, reply to query uses same #
- **flags**:
 - ❖ query or reply
 - ❖ recursion desired
 - ❖ recursion available
 - ❖ reply is authoritative



9/30/2013

Application Layer (SSL) 37

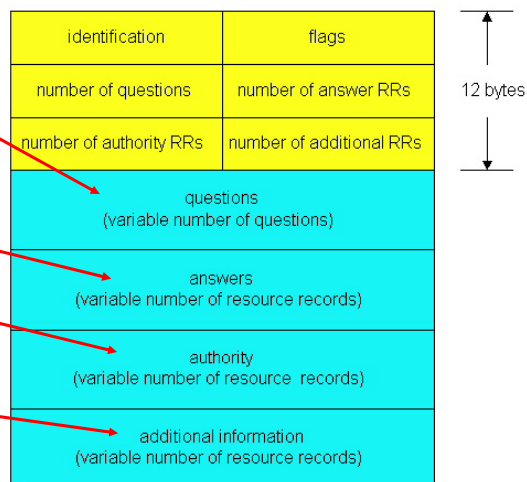
DNS protocol, messages (cont.)

Name, type fields
for a query

RRs in response
to query

records for
authoritative servers

additional "helpful"
info that may be used



9/30/2013

Application Layer (SSL) 38

Inserting records into DNS

- Example: just created startup "Network Utopia"
- Register name networkutopia.com at a **registrar** (e.g., Network Solutions)
 - ❖ Need to provide registrar with names and IP addresses of your authoritative name servers (primary and secondary)
 - ❖ Registrar inserts two RRs for each authoritative server *into .com TLD servers*:
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
- Put *into authoritative servers* Type A record for www.networkutopia.com, Type MX record for networkutopia.com, and ...
- **How do people get the IP address of your Web site?**

9/30/2013

Application Layer (SSL) 39

DDoS attacks on DNS servers

- Due to their importance, root servers, top-level-domain servers have been attacked
 - ❖ **DDoS bandwidth flooding** - root servers are protected by packet filters, which block ICMP ping messages
 - ❖ **Deluge of DNS queries to top-level domain servers** - mitigated by caching in local DNS servers
 - ❖ **Cache poisoning, attacker sends bogus replies to a DNS server**
- So far, there has not been an attack that has impeded the DNS service (yet)
 - ❖ **DNSSEC** - deployment in progress slowly

9/30/2013

Application Layer (SSL) 40

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ **2.2 Web and HTTP**
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 41

Web and HTTP

First some jargon

- ❑ Web page consists of **base HTML-file** which includes several referenced **objects**
- ❑ Object can be HTML file, JPEG image, Java applet, audio file,...
- ❑ Each object is addressable by a **URL**, for example

www.someschool.edu/someDept/pic.gif

host name

path name

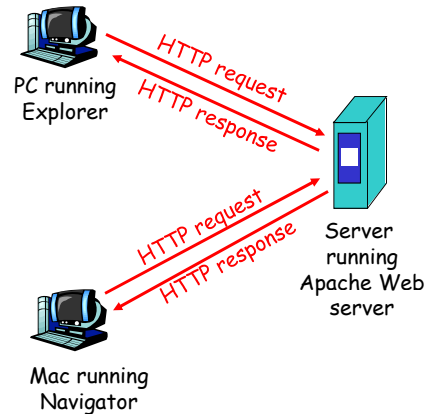
9/30/2013

Application Layer (SSL) 42

HTTP overview

HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model
 - ❖ **client**: browser that requests, receives, "displays" Web objects
 - ❖ **server**: Web server sends objects in response to requests



9/30/2013

Application Layer (SSL) 43

HTTP overview (continued)

Uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains no information about past client requests

aside
Protocols that maintain "state" are complex

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

9/30/2013

Application Layer (SSL) 44

HTTP connections

Nonpersistent HTTP

- ❑ At most one object is sent over a TCP connection.
- ❑ Version 1.0

Persistent HTTP

- ❑ Multiple objects can be sent over single TCP connection between client and server.
- ❑ Version 1.1

9/30/2013

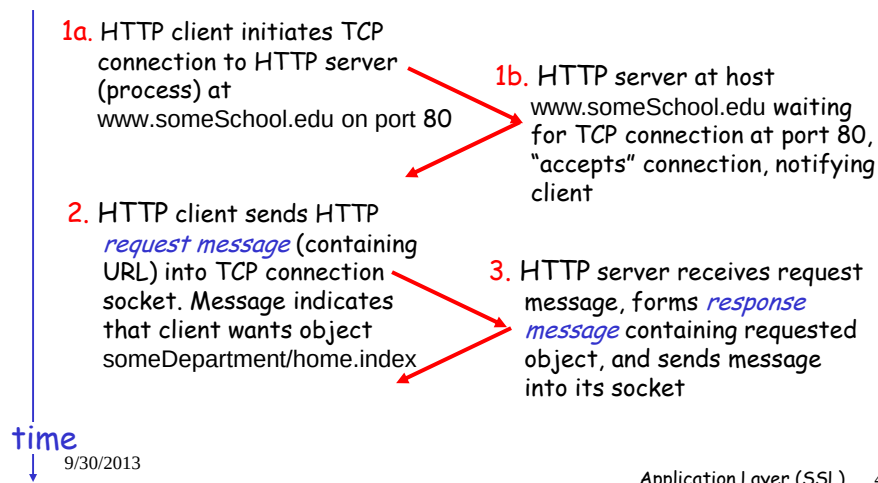
Application Layer (SSL) 45

Nonpersistent HTTP

Suppose user enters URL

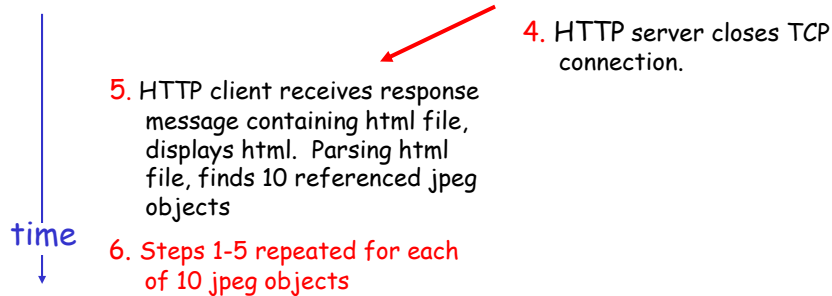
`www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)



Application Layer (SSL) 46

Nonpersistent HTTP (cont.)



9/30/2013

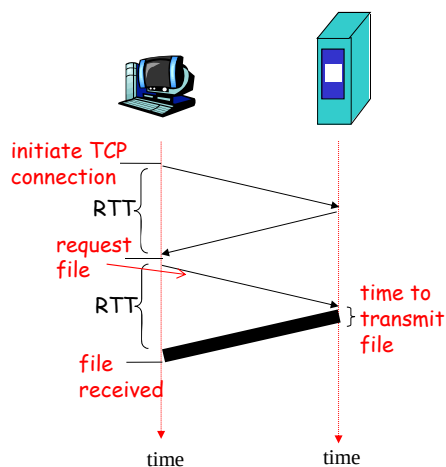
Application Layer (SSL) 47

Non-Persistent HTTP: Response time

Definition of RTT: time for a small packet to travel from client to server and back.

Response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file response time total = $2RTT + \text{file transmission time}$



9/30/2013

Application Layer (SSL) 48

Nonpersistent vs. Persistent HTTP

Nonpersistent HTTP delay

- ❑ requires 2 additional RTTs for an object
- ❑ most browsers open parallel TCP connections to fetch referenced objects
 - ❖ very little additional time for all reference objects
 - ❖ but OS overhead to allocate host resources for all TCP connections

Persistent HTTP delay

- ❑ server leaves connection open after sending response
- ❑ subsequent HTTP messages between client/server are sent over same connection
- ❑ client sends a new request as soon as it encounters a referenced object (**pipelining**)
- ❑ *as little as one additional RTT for all referenced objects (plus their transmission time)*

9/30/2013

Application Layer (SSL) 49

HTTP request message

- ❑ two types of HTTP messages: *request, response*
- ❑ **HTTP request message:**
 - ❖ ASCII (human-readable format)

request line
(GET, POST,
HEAD commands)

header
lines

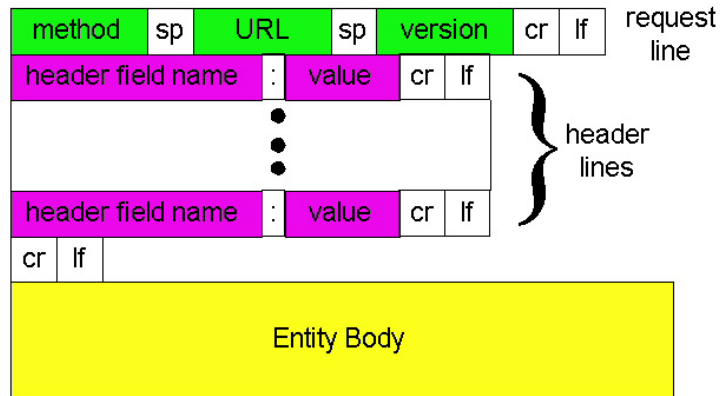
Carriage return,
line feed indicate
end of header lines

```
GET /somedir/page.html HTTP/1.1\r\n
Host: www.someschool.edu\r\n
User-agent: Mozilla/5.0\r\n
Connection: close\r\n
Accept-language: fr\r\n
\r\n
```

9/30/2013

Application Layer (SSL) 50

HTTP request message: general format



9/30/2013

Application Layer (SSL) 51

Uploading form input

Post method:

- ☐ Web page often includes form input
- ☐ Input is uploaded to server in entity body

URL method:

- ☐ Uses GET method
- ☐ Input is uploaded in URL field of request line:

www.somesite.com/animalsearch?monkeys&banana

9/30/2013

Application Layer (SSL) 52

Method types

HTTP/1.0

- ❑ GET
- ❑ POST
- ❑ HEAD
 - ❖ asks server to leave requested object out of response

HTTP/1.1

- ❑ GET, POST, HEAD
- ❑ PUT
 - ❖ uploads file in entity body to path specified in URL field
- ❑ DELETE
 - ❖ deletes file specified in the URL field
- ❑ ... (others)

9/30/2013

Application Layer (SSL) 53

HTTP response message

status line
(protocol
status code
status phrase)

header
lines

data, e.g.,
requested
HTML file

```
HTTP/1.1 200 OK\r\n
Connection close\r\n
Date: Tue, 09 Aug 2011 12:00:15 GMT\r\n
Server: Apache/2.2.3 (CentOS)\r\n
Last-Modified: Tue, 09 Aug 2011... \r\n
Content-Length: 6821\r\n
Content-Type: text/html\r\n
\r\n
```

data data data data data ...

Other methods: (2) chunks
(3) multipart/byteranges

9/30/2013

Application Layer (SSL) 54

HTTP response status codes

In first line in server->client response message.

A few sample codes:

200 OK

- ❖ request succeeded, requested object later in this message

301 Moved Permanently

- ❖ requested object moved, new location specified later in this message (Location:)

400 Bad Request

- ❖ request message not understood by server

404 Not Found

- ❖ requested document not found on this server

505 HTTP Version Not Supported

9/30/2013

Application Layer (SSL) 55

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet cis.poly.edu 80
```

Opens TCP connection to port 80 (default HTTP server port) at cis.poly.edu. Anything typed in is sent to port 80 at cis.poly.edu

2. Type in a GET HTTP request:

```
GET /~ross/ HTTP/1.1  
Host: cis.poly.edu
```

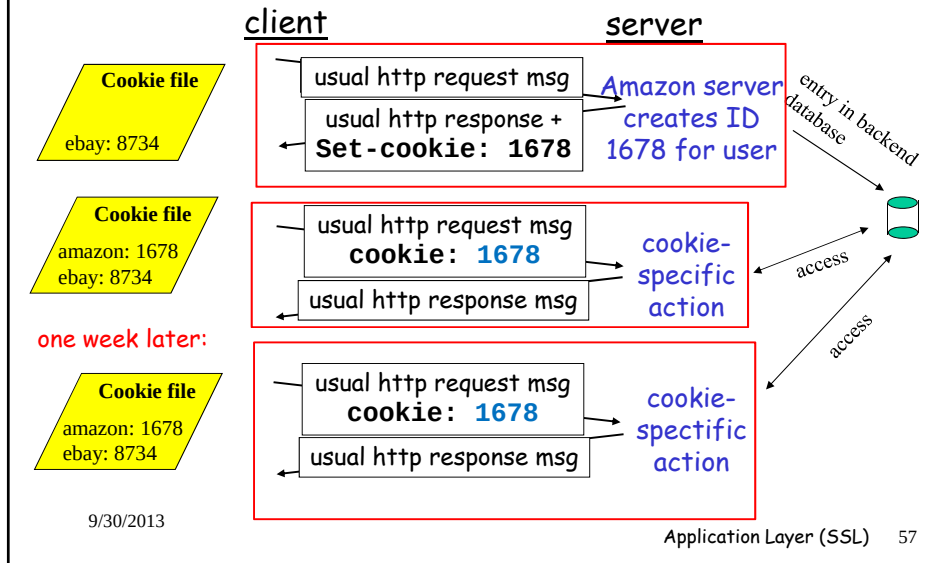
By typing this in (hit carriage return twice), you send this minimal, but complete, GET request to HTTP server

3. Look at response message sent by HTTP server

9/30/2013

Application Layer (SSL) 56

Cookies: keeping state info



Cookies

- ❑ Application end points keep state at client/server over multiple transactions
- ❑ http messages carry state info but http protocol is stateless

What cookies can bring:

- ❑ authorization
- ❑ shopping carts
- ❑ recommendations
- ❑ user session state (Web e-mail)

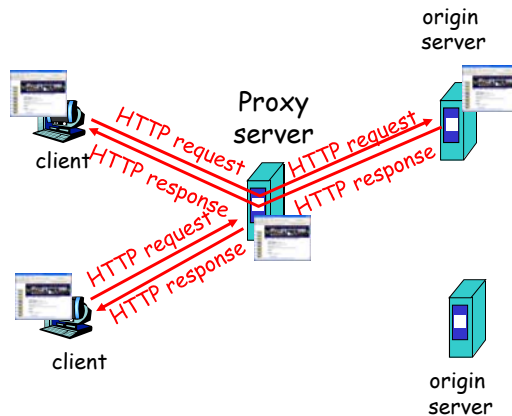
Cookies and privacy

- ❑ cookies permit sites to learn a lot about you including third parties

Web caches (proxy server)

Goal: satisfy client request without involving origin servers

- ❑ user sets browser:
Web accesses via cache
- ❑ browser sends all HTTP requests to cache
 - ❖ if object is not in cache, request object from origin server, then return object to client
 - ❖ else cache returns object



9/30/2013

Application Layer (SSL) 59

More about Web caching

- ❑ cache acts as both client and server
- ❑ typically cache is installed by ISP (university, company, residential ISP)

Why Web caching?

- ❑ reduce response time for client request
- ❑ reduce traffic on an institution's access link.

9/30/2013

Application Layer (SSL) 60

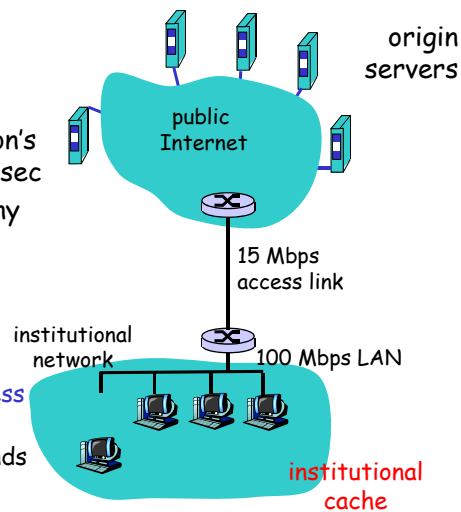
Caching example

Assumptions

- average object size = 1 Mbits
- avg. request rate from institution's browsers to origin servers = 15/sec
- Internet delay (ISP router to any origin server and back) = 2 sec

Consequences

- utilization on LAN = 15%
- utilization on access link = 100%
- total delay = Internet delay + access link delay + LAN delay
= 2 seconds + minutes + 0.02 seconds
- some lost requests



9/30/2013

Application Layer (SSL) 61

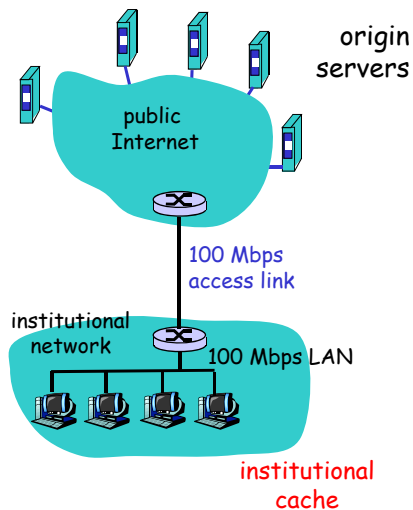
Caching example (cont)

possible solution

- increase bandwidth of access link to, say, 100 Mbps

consequence

- utilization on LAN = 15%
- utilization on access link = 15%
- Total delay = Internet delay + access link delay + LAN delay
= 2 + 0.02 + 0.02 seconds
- often a costly upgrade



9/30/2013

Application Layer (SSL) 62

Caching example (3)

possible solution: install cache

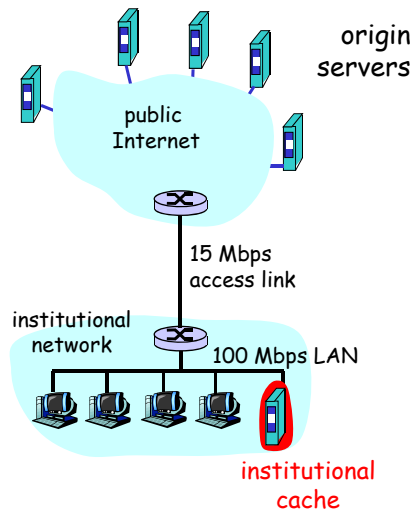
- suppose hit rate is 0.4

consequence

- 40% requests will be satisfied almost immediately
- 60% requests satisfied by origin server
- utilization of access link reduced to 60%, resulting in negligible delay (say 70 ms)
- ave. total delay = Internet delay + access link delay + LAN delay

$$= 0.6 \cdot (2 + 0.14 + 0.02) + 0.4 \cdot (0 + 0 + 0.02) \text{ seconds}$$

$$= 1.3 \text{ sec}$$



9/30/2013

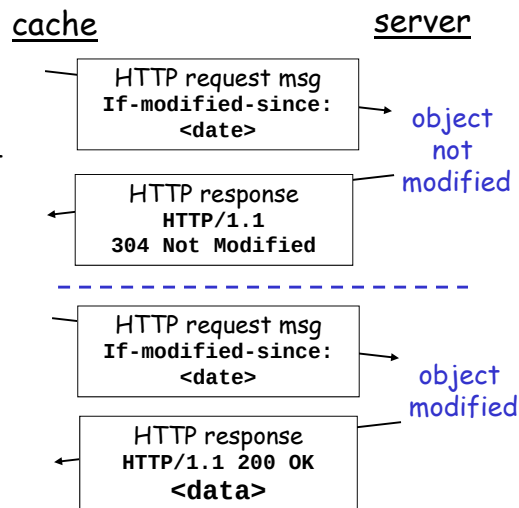
Application Layer (SSL) 63

Conditional GET

- Goal: don't send object if cache has up-to-date cached version

- cache: indicates date of cached copy in HTTP request to server
If-modified-since:
`<date>`

- server: response contains no object if cached copy is up-to-date:
HTTP/1.1 304 Not Modified



9/30/2013

Application Layer (SSL) 64

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ **2.4 Electronic Mail**
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

9/30/2013

Application Layer (SSL) 65

Electronic Mail

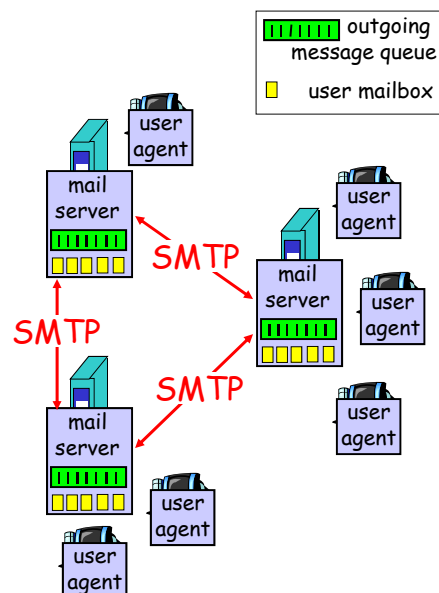
Three major components:

- ❑ user agents
- ❑ mail servers
- ❑ simple mail transfer protocol: SMTP

User Agent

- ❑ a.k.a. "mail reader"
- ❑ composing, editing, reading mail messages
- ❑ e.g., Outlook, alpine, Mozilla Thunderbird, iPhone mail client
- ❑ outgoing, incoming messages stored on server

(Let's ignore web mail for a little while)



9/30/2013

Application Layer (SSL) 66

Electronic Mail: mail servers

Mail Servers

- ❑ **mailbox** contains incoming messages for user
- ❑ **message queue** of outgoing (to be sent) mail messages

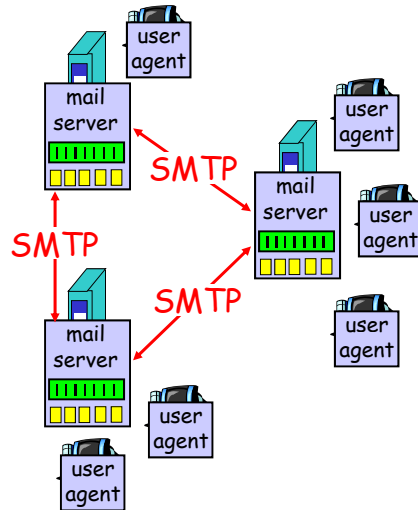
SMTP protocol

between mail servers to send email messages

- ❖ "client": sender
- ❖ "server": receiver

Looks like it is the very first (hybrid) P2P system!

9/30/2013



Application Layer (SSL) 67

Electronic Mail: SMTP

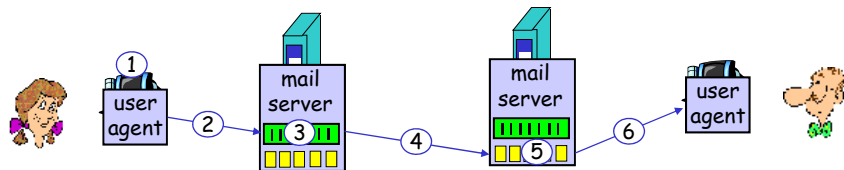
- ❑ uses TCP to reliably transfer email message from client to server, port 25
- ❑ direct transfer: sending server to receiving server
- ❑ three phases of transfer
 - ❖ handshaking (greeting)
 - ❖ transfer of messages (persistent connection)
 - ❖ closure
- ❑ command/response interaction
 - ❖ **commands**: ASCII text
 - ❖ **response**: status code and phrase
- ❑ messages must be in **7-bit ASCII**

9/30/2013

Application Layer (SSL) 68

Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message to **bob@someschool.edu**
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



9/30/2013

Application Layer (SSL) 69

Sample smtp interaction

(after TCP connection established)

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

9/30/2013

Application Layer (SSL) 70

Try SMTP interaction for yourself:

- ❑ `telnet servername 25`
- ❑ see 220 reply from server
- ❑ enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

Try utcs mail server, mail.cs.utexas.edu

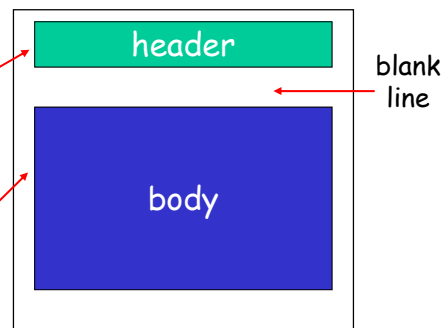
9/30/2013

Application Layer (SSL) 71

Mail message format

SMTP (rfc 5321): protocol for exchanging email msgs
rfc 5322: standard for text message format:

- ❑ header lines, e.g.,
 - ❖ To:
 - ❖ From:
 - ❖ Subject:*different from SMTP commands*
- ❑ body
 - ❖ the "message", ASCII characters only

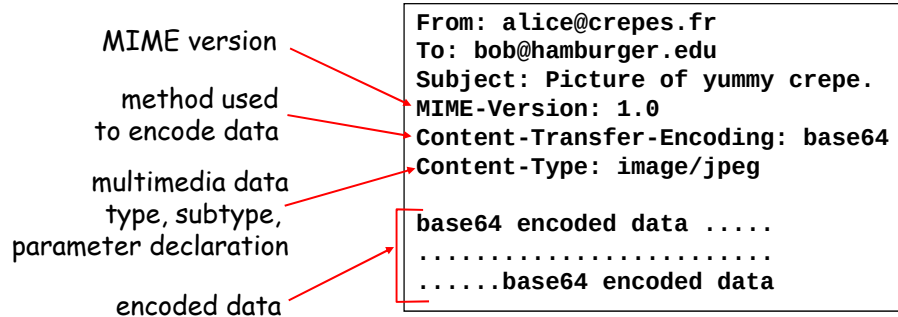


9/30/2013

Application Layer (SSL) 72

Message format: multimedia extensions

- ❑ Multipurpose Internet Mail Extensions (MIME)
- ❑ additional lines in msg header declare MIME content type



9/30/2013

Application Layer (SSL) 73

smtp: final words

- ❑ smtp uses persistent connections
- ❑ smtp requires that message (header & body) be in 7-bit ascii
- ❑ smtp server uses **CRLF.CRLF** to determine end of message
- ❑ certain character strings are not permitted in message (e.g., **CRLF.CRLF**). Thus message has to be encoded (e.g., into base-64 or quoted printable)

Comparison with http

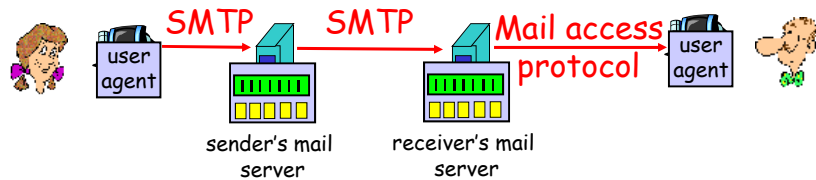
- ❑ http: pull
- ❑ email: push
- ❑ both have ASCII command/response interaction, status codes
- ❑ http: each object is encapsulated in its own response message
- ❑ smtp: multiple objects in a message sent in a *multipart* message

Content-Type: multipart/alternative; boundary=089e01177475fd94ce04e7779c43

9/30/2013

Application Layer (SSL) 74

Mail access protocols



- SMTP: delivery/storage to receiver's server
- Mail access protocol: retrieval from server
 - ❖ POP: Post Office Protocol [RFC 1939]
 - authorization (agent <-->server) and download
 - ❖ IMAP: Internet Mail Access Protocol [RFC 3501]
 - more features (more complex)
 - manipulation of stored msgs in folders on server
- HTTP: gmail, Hotmail, Yahoo! Mail, etc.

9/30/2013

Application Layer (SSL) 75

Chapter 2: Application layer

- | | |
|--|-----------------------------------|
| □ 2.1 Principles of network applications | □ 2.6 P2P applications |
| □ 2.2 Web and HTTP | □ 2.7 Socket programming with TCP |
| □ 2.3 FTP | □ 2.8 Socket programming with UDP |
| □ 2.4 Electronic Mail | |
| □ 2.5 DNS | |

9/30/2013

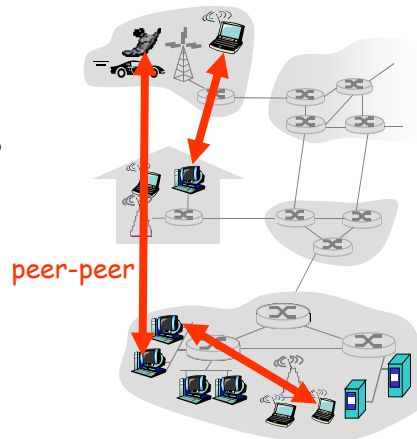
Application Layer (SSL) 76

P2P architecture

- no always-on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses

□ Examples:

- ❖ File distribution, e.g., BitTorrent (*has trackers*)
- ❖ Internet telephony, e.g., Skype (*has super peers*),

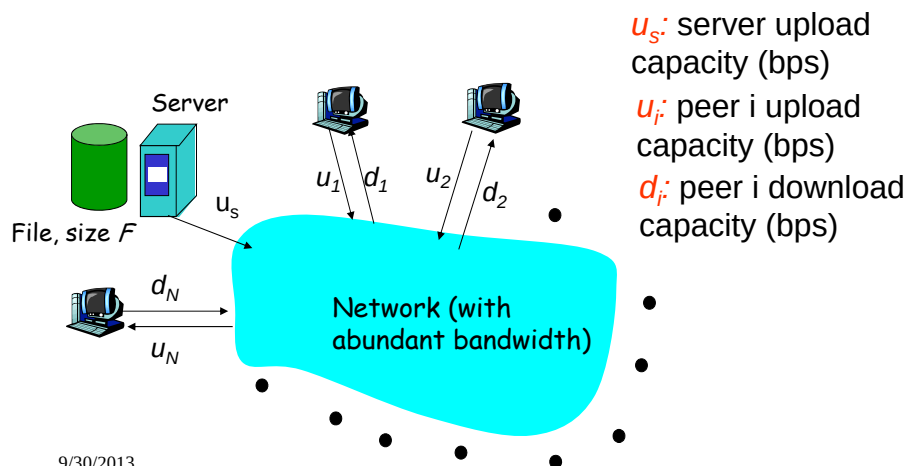


9/30/2013

Application Layer (SSL) 77

File Distribution: Client-Server vs P2P

Question: How much time to distribute file from one server to N peers?

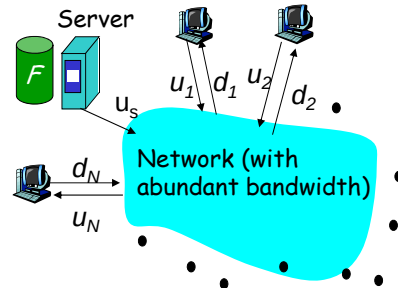


9/30/2013

Application Layer (SSL) 78

File distribution time: Client-Server

- server sequentially sends N copies:
 - ❖ NF/u_s time
- client i takes F/d_i time to download



Time to distribute F
to N clients
lower bound

$$= d_{cs} \geq \max \{ NF/u_s, F/\min_i(d_i) \}$$

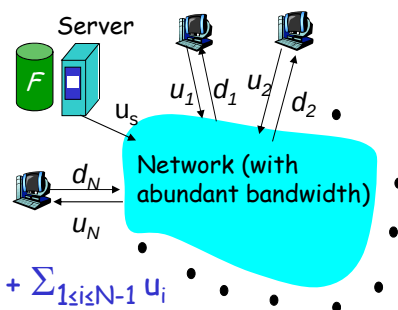
increases linearly in N
(this term for large N)

9/30/2013

Application Layer (SSL) 79

File distribution time: P2P

- server must send one copy: F/u_s time
- client i takes F/d_i time to download
- NF bits must be uploaded (aggregate)



fastest possible upload rate: $u_s + \sum_{1 \leq i \leq N-1} u_i$

$$d_{p2p} \geq \max \{ F/u_s, F/\min_i(d_i), NF/(u_s + \sum u_i) \}$$

Largest value as $N \uparrow$

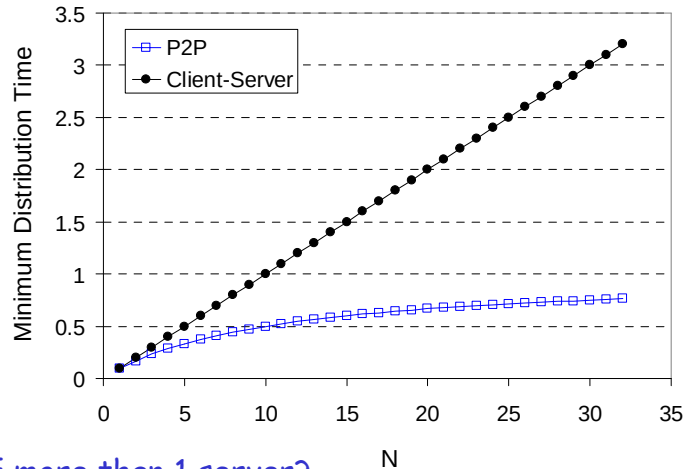
for large N , $u_s > (u_s + \sum u_i)/N$ and $\min_i(d_i) > (u_s + \sum u_i)/N$

9/30/2013

Application Layer (SSL) 80

Client-server vs. P2P: example

Client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{\min} \geq u_s$



What if more than 1 server?

9/30/2013

Application Layer (SSL) 81

Chapter 2: Summary

our study of network apps now complete!

- application architectures
 - ❖ client-server
 - ❖ P2P
 - ❖ hybrid
- application service requirements:
 - ❖ reliability, bandwidth, delay
- Internet transport service model
 - ❖ connection-oriented, reliable: TCP
 - ❖ unreliable, datagrams: UDP
- socket programming
- specific protocols:
 - ❖ HTTP
 - ❖ FTP
 - ❖ SMTP
 - ❖ DNS
 - ❖ P2P: BitTorrent, Skype

9/30/2013

Application Layer (SSL) 82

Chapter 2: Summary

Most importantly: learned about *protocols*

- typical request/reply message exchange:

- ❖ client requests info or service
- ❖ server responds with data, status code

- message formats:

- ❖ header: fields encode protocol control info
- ❖ data: info (payload) being communicated

Important themes:

- control vs. data info
- centralized vs. decentralized
- stateless vs. stateful
- reliable vs. unreliable msg transfer
- "complexity at network edge"

9/30/2013

Application Layer (SSL) 83

End of Chapter 2

9/30/2013

Application Layer (SSL) 84